

Domain-Specific Question Answering System with Agentic Verification and Explainable Responses

Course: M.Tech Artificial Intelligence and Machine Learning — Natural Language Processing Applications

Assignment: NLP Applications Project Assignment 1

Student Details

Group 11 NLP

Student name	Student ID
Kodhandan S	2024ac05203
Gowrishankar S	2024ac05046
Arunkumar K A	2024ac05045
Naveen Rajendiran	2024ac05219
Kumaresh Babu A	2024ac05304

Problem Statement

The objective of this assignment is to develop a **Domain-Specific Question Answering (QA) System** that not only answers questions accurately but also verifies the generated answers and explains the reasoning behind them. The system integrates Generative AI techniques with an Agentic Verification loop to ensure grounded, trustworthy, and explainable responses.

The assignment is structured into five tasks:

- Task 1:** Domain Selection and Knowledge Base Creation
 - Task 2:** Question Classification and Intent Understanding
 - Task 3:** Answer Generation Using Generative AI Techniques
 - Task 4:** Agentic Verification and Self-Correction
 - Task 5:** System Evaluation, Limitations, and Encoder-Only Architecture Discussion
-

Dataset / Corpus Description

The knowledge base for this system is constructed from **five synthetic medical diagnosis support documents**, each modelled on authoritative clinical guidelines published by organisations such as the World Health Organization (WHO), the Centers for Disease Control and Prevention (CDC), and established medical textbooks.

The five documents cover the following medical conditions:

Document ID	Medical Condition	Focus Areas
D1	Diabetes Mellitus	Types, symptoms, diagnostic criteria, treatment
D2	Hypertension	Causes, classification, pharmacological management
D3	COVID-19	Transmission, symptom severity, treatment protocols
D4	Bronchial Asthma	Triggers, classification, inhaler therapy
D5	Acute Appendicitis	Symptoms, diagnosis, surgical intervention

Each document is segmented into thematic paragraphs (chunks), enabling precise retrieval during the QA pipeline.

Tools and Libraries Used

Library / Tool	Version	Purpose
<code>transformers</code>	≥4.35	Flan-T5 generation, BART zero-shot classification
<code>sentence-transformers</code>	≥2.2	Semantic embeddings for retrieval
<code>torch</code>	≥2.0	Deep learning backend
<code>scikit-learn</code>	≥1.2	Cosine similarity computation
<code>pandas</code>	≥1.5	Tabular display of results
<code>numpy</code>	≥1.24	Numerical operations
<code>IPython</code>	—	Rich notebook display

```
import sys
import subprocess

# Install required libraries into the active kernel's environment
```

```
packages = [
    "numpy", "pandas", "scikit-learn",
    "torch", "transformers", "sentence-transformers"
]
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q"] +
packages)

print("All packages installed. Ready to import.")

All packages installed. Ready to import.

# Global Imports
import re
import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer
from transformers import pipeline, T5ForConditionalGeneration,
T5Tokenizer
from transformers.utils import logging
from IPython.display import display, Markdown
import warnings
warnings.filterwarnings('ignore')

print("All libraries imported successfully.")

All libraries imported successfully.
```

Task 1: Domain Selection and Knowledge Base Creation

Marks: 2

Aim

The aim of this task is to select an appropriate specialised domain for building a Question Answering system, curate a structured knowledge base from domain-specific documents, and formally define the system's purpose, users, and associated risks. The task establishes the foundational corpus upon which all subsequent modules — retrieval, generation, and verification — will operate.

1.1 Domain Definition

Parameter	Details
Domain Selected	Medical Diagnosis Support Documents
Purpose of the QA System	To provide accurate, context-grounded answers to clinical queries pertaining to disease symptoms, diagnostic criteria, and treatment protocols, thereby supporting clinical decision-making and medical education
Knowledge Base Source	Synthetic clinical documents modelled on WHO guidelines, CDC clinical protocols, and standard medical textbook chapters covering five prevalent medical conditions
Target Users	Junior physicians and interns requiring quick clinical reference, medical students during case-based learning, clinical decision support system developers, and healthcare educators
Expected Risks if System Gives Wrong Answer	Misdiagnosis leading to inappropriate treatment; prescription of contraindicated medications; delayed referral to specialist care; patient harm or mortality in emergency conditions; medicolegal liability for clinical staff relying on the system

1.2 Knowledge Base Construction

```
# -----  
# KNOWLEDGE BASE – 5 Medical Diagnosis Support Documents  
# Each document is a dictionary with title and full text content.  
# -----  
  
raw_documents = [  
    {  
        "doc_id": "D1",  
        "title": "Diabetes Mellitus – Clinical Overview",  
        "text": ""  
        Diabetes mellitus is a chronic metabolic disorder characterised by  
persistent hyperglycaemia resulting  
from defects in insulin secretion, insulin action, or both. It is  
classified primarily into Type 1,  
Type 2, and gestational diabetes.  
  
Type 1 diabetes mellitus is an autoimmune condition in which the  
pancreatic beta cells are destroyed  
by the body's own immune system, leading to absolute insulin  
deficiency. It predominantly affects  
children and young adults. Patients require lifelong exogenous insulin
```

therapy for survival.

The onset is typically acute with symptoms of polyuria, polydipsia, polyphagia, and unexplained weight loss.

Type 2 diabetes mellitus is the most prevalent form, accounting for approximately 90–95% of all diabetes cases. It is characterised by insulin resistance in peripheral tissues and a progressive decline in beta cell function. Risk factors include obesity, sedentary lifestyle, family history, and age above 45 years. Initial management involves lifestyle modifications such as dietary adjustments and exercise. Pharmacological therapy typically begins with metformin, followed by additional oral hypoglycaemic agents or insulin as required.

Gestational diabetes mellitus occurs during pregnancy when hormonal changes impair insulin action.

It increases the risk of complications for both the mother and neonate, including macrosomia, preterm birth, and increased risk of Type 2 diabetes later in life.

Diagnostic criteria for diabetes mellitus, as per the American Diabetes Association (ADA), include:

fasting plasma glucose ≥ 126 mg/dL, 2-hour plasma glucose ≥ 200 mg/dL during an oral glucose tolerance test (OGTT), HbA1c $\geq 6.5\%$, or random plasma glucose ≥ 200 mg/dL with classic hyperglycaemic symptoms.

Long-term complications of uncontrolled diabetes include diabetic retinopathy, nephropathy, peripheral neuropathy, cardiovascular disease, and diabetic foot ulcers. Regular monitoring of HbA1c, blood pressure, lipid profile, and renal function is essential for complication prevention.

"""

```
},  
{
```

```
    "doc_id": "D2",  
    "title": "Hypertension – Clinical Overview",  
    "text": ""
```

Hypertension, commonly referred to as high blood pressure, is defined as a sustained elevation of systolic blood pressure ≥ 130 mmHg or diastolic blood pressure ≥ 80 mmHg, as per the 2017 ACC/AHA guidelines. It is one of the most significant modifiable risk factors for cardiovascular disease, stroke, and renal failure.

Hypertension is classified into primary (essential) hypertension, which accounts for 90–95% of cases and has no identifiable single cause, and secondary hypertension, which results from identifiable conditions such as renal artery stenosis, primary aldosteronism, or pheochromocytoma.

The staging of hypertension is as follows: Normal blood pressure is below 120/80 mmHg.

Elevated blood pressure ranges from 120–129 systolic with diastolic below 80 mmHg.

Stage 1 hypertension is defined as 130–139/80–89 mmHg, and Stage 2 hypertension is

140/90 mmHg or higher. A hypertensive crisis is defined as blood pressure exceeding

180/120 mmHg and requires immediate medical intervention.

Common symptoms of hypertension include headache, dizziness, blurred vision, and shortness of breath. However, hypertension is often asymptomatic and is called the 'silent killer' because organ damage can progress without noticeable symptoms.

Non-pharmacological management strategies include the DASH diet (Dietary Approaches to Stop

Hypertension), sodium restriction to below 2.3 g/day, regular aerobic exercise, weight

reduction, smoking cessation, and moderation of alcohol intake.

First-line pharmacological agents for hypertension include thiazide diuretics,

angiotensin-converting enzyme (ACE) inhibitors, angiotensin receptor blockers (ARBs),

and calcium channel blockers. Beta-blockers are recommended in patients with concurrent

heart failure or ischaemic heart disease.

"""

```
},  
{
```

```
    "doc_id": "D3",  
    "title": "COVID-19 – Clinical Management and Treatment  
Protocols",  
    "text": ""
```

Coronavirus Disease 2019 (COVID-19) is an infectious disease caused by the SARS-CoV-2 virus,

first identified in Wuhan, China, in December 2019. The virus is primarily transmitted through

respiratory droplets and aerosols produced when an infected person

coughs, sneezes, or speaks.

The clinical presentation of COVID-19 ranges from asymptomatic infection to critical illness. Common symptoms include fever, dry cough, fatigue, loss of taste or smell (anosmia/ageusia), sore throat, headache, and myalgia. Severe disease is characterised by dyspnoea, hypoxaemia ($\text{SpO}_2 < 94\%$), and bilateral pneumonia on chest imaging.

Disease severity is classified as mild (symptoms without dyspnoea), moderate (lower respiratory involvement, $\text{SpO}_2 \geq 94\%$), severe ($\text{SpO}_2 < 94\%$, respiratory rate $> 30/\text{min}$), and critical (respiratory failure, septic shock, or multi-organ dysfunction).

High-risk groups include elderly patients, individuals with diabetes, hypertension, obesity, chronic lung disease, and immunocompromised individuals.

Treatment for mild cases is primarily supportive: rest, adequate hydration, antipyretics for fever management, and self-isolation. Antiviral agents such as nirmatrelvir-ritonavir (Paxlovid) are recommended for high-risk patients with mild-to-moderate disease within 5 days of symptom onset.

Hospitalised patients with severe COVID-19 receive supplemental oxygen therapy, dexamethasone (6 mg/day for up to 10 days), and remdesivir for patients requiring oxygen supplementation. Tocilizumab or baricitinib is recommended for patients with rapidly progressing disease. Thromboprophylaxis with low-molecular-weight heparin is standard care to prevent COVID-19-associated coagulopathy.

"""

```
},  
{
```

```
    "doc_id": "D4",  
    "title": "Bronchial Asthma – Diagnosis and Management",  
    "text": ""
```

Bronchial asthma is a chronic inflammatory disease of the airways characterised by variable and recurring symptoms of airflow obstruction and bronchospasm. It affects approximately 300 million people worldwide and is among the most common chronic respiratory diseases in both children and adults.

Asthma symptoms include episodic wheezing, breathlessness, chest tightness, and cough, particularly at night or in the early morning. These symptoms are typically triggered by allergens (dust mites, pollen, pet dander), respiratory infections, cold air, exercise, smoke, and air pollutants.

According to the GINA (Global Initiative for Asthma) guidelines, asthma is classified by symptom control into well-controlled, partly controlled, and uncontrolled categories. Severity is further assessed based on the intensity of treatment required to achieve control.

Diagnosis is confirmed by spirometry demonstrating a post-bronchodilator improvement in FEV₁ of $\geq 12\%$ and ≥ 200 mL, indicating reversible airflow obstruction. Peak expiratory flow (PEF) monitoring is also used in clinical settings.

The pharmacological management of asthma follows a stepwise approach. Short-acting beta-2 agonists (SABAs), such as salbutamol (albuterol), are the first-line reliever inhalers for acute symptom relief. Inhaled corticosteroids (ICS), such as beclomethasone and budesonide, are the cornerstone of controller therapy.

Long-acting beta-2 agonists (LABAs) such as formoterol and salmeterol are added as combination therapy with ICS for patients with persistent asthma not adequately controlled by ICS alone.

Leukotriene receptor antagonists (montelukast) provide an alternative add-on therapy.

Biological agents such as mepolizumab and omalizumab are reserved for severe, refractory asthma.

"""

```
},  
{
```

```
    "doc_id": "D5",  
    "title": "Acute Appendicitis – Diagnosis and Surgical  
Management",  
    "text": ""
```

Acute appendicitis is the inflammation of the vermiform appendix and is one of the most common causes of acute abdominal pain requiring emergency surgical intervention. It has a lifetime prevalence of approximately 7–8% and predominantly affects individuals

between 10 and 30 years of age.

The pathophysiology of acute appendicitis involves obstruction of the appendiceal lumen, most commonly by a faecolith (hardened faecal matter), followed by bacterial overgrowth, distension, ischaemia, and eventual perforation if left untreated.

Classic clinical features include periumbilical pain that migrates to the right iliac fossa (McBurney's point), accompanied by nausea, vomiting, fever, and anorexia. Rebound tenderness, guarding, and Rovsing's sign are positive on physical examination in advanced cases.

The Alvarado scoring system is used to stratify the likelihood of acute appendicitis. A score of 7–10 is indicative of probable appendicitis and warrants surgical exploration. Diagnostic workup includes a complete blood count (leukocytosis with neutrophilia), C-reactive protein (CRP), and imaging – abdominal ultrasound is the first-line modality, with contrast-enhanced CT abdomen used when ultrasound is inconclusive.

The definitive treatment for acute appendicitis is appendicectomy (appendectomy), performed either via open or laparoscopic approach. Laparoscopic appendicectomy is the preferred technique due to reduced postoperative pain, shorter hospital stay, and lower wound infection rates. Pre-operative intravenous antibiotics (cefuroxime and metronidazole) are administered to reduce the risk of surgical site infection.

Conservative antibiotic management (with amoxicillin-clavulanate) may be considered in uncomplicated appendicitis in select patients. However, recurrence rates of 20–40% over 5 years necessitate careful patient selection and follow-up.

```
"""
    }
]

print(f"Total documents loaded: {len(raw_documents)}")
for doc in raw_documents:
    print(f" [{doc['doc_id']}] {doc['title']}")
```

```
Total documents loaded: 5
[D1] Diabetes Mellitus – Clinical Overview
[D2] Hypertension – Clinical Overview
[D3] COVID-19 – Clinical Management and Treatment Protocols
[D4] Bronchial Asthma – Diagnosis and Management
[D5] Acute Appendicitis – Diagnosis and Surgical Management
```

```
# -----
# CHUNKING – Split each document into paragraph-level chunks
# Each chunk becomes an independently retrievable unit.
# -----
```

```
def chunk_document(doc):
    """Split document text into non-empty paragraph chunks."""
    paragraphs = [p.strip() for p in doc["text"].split("\n\n") if
p.strip()]
    chunks = []
    for idx, para in enumerate(paragraphs):
        chunks.append({
            "doc_id": doc["doc_id"],
            "title": doc["title"],
            "chunk_id": f"{doc['doc_id']}_C{idx+1}",
            "text": para
        })
    return chunks
```

```
# Build the knowledge base
knowledge_base = []
for doc in raw_documents:
    knowledge_base.extend(chunk_document(doc))
```

```
print(f"Total chunks in Knowledge Base: {len(knowledge_base)}")
print()
```

```
# Summary table
summary_data = []
for doc in raw_documents:
    doc_chunks = [c for c in knowledge_base if c["doc_id"] ==
doc["doc_id"]]
    word_count = len(doc["text"].split())
    summary_data.append({
        "Doc ID": doc["doc_id"],
        "Title": doc["title"],
        "Number of Chunks": len(doc_chunks),
        "Word Count": word_count
    })
```

```
summary_df = pd.DataFrame(summary_data)
print("Knowledge Base Summary:")
display(summary_df)
```

Total chunks in Knowledge Base: 30

Knowledge Base Summary:

```
{"summary":{"\n  \"name\": \"summary_df\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"Doc ID\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"D2\",\n          \"D5\",\n          \"D3\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Title\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"Hypertension \\\u2014 Clinical Overview\",\n          \"Acute Appendicitis \\\u2014 Diagnosis and Surgical Management\",\n          \"COVID-19 \\\u2014 Clinical Management and Treatment Protocols\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Number of Chunks\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 6,\n        \"max\": 6,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          6\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Word Count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 23,\n        \"min\": 217,\n        \"max\": 281,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          241\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  },\n  \"type\": \"dataframe\",\n  \"variable_name\": \"summary_df\"}
```

Display a sample chunk to verify structure

```
print("Sample Chunk from Knowledge Base:")
print("-" * 60)
sample = knowledge_base[2]
print(f"Chunk ID    : {sample['chunk_id']}")
print(f"Document     : {sample['title']}")
print(f"Text          : {sample['text'][:300]}...")
```

Sample Chunk from Knowledge Base:

```
-----
Chunk ID    : D1_C3
Document    : Diabetes Mellitus – Clinical Overview
Text        : Type 2 diabetes mellitus is the most prevalent form,
accounting for approximately 90–95% of all
diabetes cases. It is characterised by insulin resistance in
peripheral tissues and a progressive
decline in beta cell function. Risk factors include obesity, sedentary
lifestyle, family history,
and age ...
```

Explanation of Method Used

We split each document into individual paragraphs instead of treating the whole document as one big chunk. The idea is simple — if someone asks a specific question like "what is the glucose threshold for diabetes?", we don't want to retrieve an entire 500-word document. We want just the paragraph that actually talks about diagnostic criteria. Smaller chunks give more precise results.

Why we used paragraph-level chunks: Each paragraph in a clinical document usually talks about one thing — either the causes, or the symptoms, or the treatment. Keeping those boundaries makes sure we're not mixing unrelated information into a single chunk. This made retrieval noticeably cleaner when we tested it.

What we also considered: We thought about splitting by fixed token size (like 128 tokens with some overlap), which is a common approach. But the problem with that in medical text is it can cut a sentence in the middle — for example, a sentence saying "fasting glucose ≥ 126 mg/dL is the diagnostic threshold" might get split between two chunks and lose its meaning. So we stuck with paragraph splitting.

Inference

The knowledge base ended up with around 25–30 chunks across the five documents. That feels like a good amount — not too few that retrieval becomes vague, and not so many that it slows things down. Since the five conditions cover very different medical areas (metabolic, cardiac, infectious, respiratory, surgical), it also gave us a good way to check whether retrieval actually picks the right document and doesn't confuse, say, a diabetes question with a COVID question.

Why Medical QA Systems Need to Be Accurate and Grounded

In most domains, a wrong answer is just inconvenient. In medicine, it can cause real harm. If the system says the diabetes diagnostic threshold is 100 mg/dL instead of 126 mg/dL, a clinician relying on it might misclassify a patient. That's a serious problem.

- **Accuracy** matters because medical decisions are based on exact numbers — dosages, thresholds, lab values. Getting these wrong is not a minor issue.
- **Grounding** means every answer should be traceable back to a source. It's not enough for the answer to sound right — it should come from something the system actually retrieved.
- **Verification** is needed because language models hallucinate. They can generate confident-sounding answers that are completely wrong. In a medical system, that's dangerous precisely *because* it sounds credible.

Why Agentic AI Helps in High-Risk Systems

Normal generative models just produce one answer and stop. An agentic system goes further — it checks its own answer, decides if it's good enough, and tries again if not. This is useful in medicine because a single wrong response can have consequences that can't be undone.

In this system, the agent can flag answers that aren't backed by the retrieved context, try to get better context if needed, and say "I don't know" when the question is outside the scope of the knowledge base. That kind of self-awareness is important when the system is being used for something that actually matters.

Task 2: Question Classification and Intent Understanding

Marks: 2

Aim

The aim of this task is to implement a question classification module that categorises incoming user queries into semantically meaningful types — Factual, Definition-based, Procedural, Comparison-based, Reasoning-based, Ambiguous, and Out-of-context. This module functions as a routing layer in the QA pipeline, enabling the downstream answer generation module to adapt its strategy based on the nature of the question.

```
# 

---


# TASK 2 – STEP 1: Rule-Based Question Classifier
# Uses lexical pattern matching to assign a preliminary label.
# 

---



QUESTION_TYPES = [
    "Factual",
    "Definition-based",
    "Procedural",
    "Comparison-based",
    "Reasoning-based",
    "Ambiguous",
    "Out-of-context"
]

# Medical domain keywords for boundary detection
MEDICAL_KEYWORDS = [
    "diabetes", "hypertension", "covid", "asthma", "appendicitis",
    "blood pressure", "insulin", "glucose", "fever", "symptom",
    "treatment", "diagnosis", "medication", "inhaler", "surgery",
    "appendix", "infection", "vaccine", "disease", "patient",
    "hbalc", "fev1", "sars", "respiratory", "cardiac", "renal",
    "metformin", "steroid", "antibiotic", "oxygen", "dexamethasone"
]
```

```

def rule_based_classify(question: str) -> tuple:
    """
    Rule-based classifier using keyword and syntactic pattern
    matching.
    Returns (predicted_type, reason_string).
    """
    q = question.lower().strip()
    is_medical = any(kw in q for kw in MEDICAL_KEYWORDS)

    # Pattern matching rules in priority order
    if re.search(r"\bwhat is\b|\bdefine\b|\bmeaning of\b|\bwhat do you
mean\b|\bexplain what\b", q):
        return "Definition-based", "Contains definitional trigger
phrase (what is / define)"

    if re.search(r"\bhow (to|do|should|can|is it possible)\b|\bsteps
(to|for)\b|\bprocedure\b|\bprocess\b|\bprotocol\b|\bmanage\b|\btreat\b", q):
        return "Procedural", "Contains procedural trigger phrase (how
to / steps / procedure / treat)"

    if re.search(r"\bdifference between\b|\bcompare\b|\bversus\b|\bvs\
.?\b|\bcontrast\b|\bsimilar\b|\bbetter than\b", q):
        return "Comparison-based", "Contains comparative trigger
phrase (difference / compare / versus)"

    if re.search(r"\bwhy\b|\breason\b|\bcause\b|\bexplain\b|\bhow
does\b|\bwhat causes\b|\bpathophysiology\b", q):
        return "Reasoning-based", "Contains causal/explanatory trigger
phrase (why / reason / cause / explain)"

    if re.search(r"\bwho\b|\bwhen\b|\bwhich\b|\bhow many\b|\bhow much\b|\bwhat
(are|is the)\b|\bname\b", q):
        if is_medical:
            return "Factual", "WH-question with specific medical
referent (who/when/which/how many)"

        if not is_medical:
            return "Out-of-context", "No medical domain keywords detected;
question is outside the knowledge base scope"

    return "Ambiguous", "Medical topic detected but question intent is
unclear or underspecified"

print("Rule-based classifier loaded.")

```

Rule-based classifier loaded.

```

#
# TASK 2 – STEP 2: Zero-Shot NLI-Based Classifier

```

```

# Uses facebook/bart-large-mnli for zero-shot classification.
# The NLI head acts as an encoder-style intent classifier.
# -----

from transformers import pipeline # safe to re-import if cell run
standalone

print("Loading zero-shot classification model (facebook/bart-large-
mnli)...")
print("This may take a few minutes on first run.")

logging.disable_progress_bar()
zs_classifier = pipeline(
    "zero-shot-classification",
    model="facebook/bart-large-mnli",
    device=-1 # Use CPU; set device=0 if GPU is available
)

candidate_labels = [
    "factual question about a specific medical value or statistic",
    "definition or explanation of a medical term or concept",
    "procedural question about how to treat or diagnose",
    "comparison between two medical conditions or treatments",
    "reasoning question about causes or mechanisms of disease",
    "ambiguous or unclear medical question",
    "question unrelated to medicine or healthcare"
]

label_map = {
    "factual question about a specific medical value or statistic":
    "Factual",
    "definition or explanation of a medical term or concept":
    "Definition-based",
    "procedural question about how to treat or diagnose":
    "Procedural",
    "comparison between two medical conditions or treatments":
    "Comparison-based",
    "reasoning question about causes or mechanisms of disease":
    "Reasoning-based",
    "ambiguous or unclear medical question": "Ambiguous",
    "question unrelated to medicine or healthcare": "Out-of-context"
}

def zeroshot_classify(question: str) -> tuple:
    """Returns (predicted_type, confidence_score)."""
    result = zs_classifier(question, candidate_labels)
    top_label = result["labels"][0]
    top_score = result["scores"][0]
    return label_map[top_label], round(top_score, 3)

```

```
print("Zero-shot classifier loaded successfully.")
```

Loading zero-shot classification model (facebook/bart-large-mnli)...
This may take a few minutes on first run.
Zero-shot classifier loaded successfully.

```
# -----  
# TASK 2 – STEP 3: Hybrid Classification  
# Rule-based provides the primary classification; zero-shot  
# validates or overrides when rule confidence is low.  
# -----
```

```
def hybrid_classify(question: str) -> dict:  
    """Combine rule-based and zero-shot classification."""  
    rule_type, rule_reason = rule_based_classify(question)  
    zs_type, zs_confidence = zeroshot_classify(question)  
  
    # If rule-based returns Ambiguous, defer to zero-shot model  
    if rule_type == "Ambiguous" and zs_confidence >= 0.50:  
        final_type = zs_type  
        final_reason = f"Rule-based: Ambiguous; Zero-shot override:  
{zs_type} (confidence={zs_confidence})"  
    else:  
        final_type = rule_type  
        final_reason = f"{rule_reason} [ZS confirmation: {zs_type},  
score={zs_confidence}]"  
  
    return {  
        "Question": question,  
        "Predicted Type": final_type,  
        "Reason": final_reason  
    }
```

```
print("Hybrid classifier ready.")
```

Hybrid classifier ready.

```
# -----  
# TASK 2 – TEST: Classify 10 questions covering all 7 types  
# -----
```

```
test_questions = [  
    "What is diabetes mellitus?",  
    # Definition-based  
    "What is the fasting plasma glucose threshold for diagnosing  
diabetes?", # Factual  
    "How should acute appendicitis be treated surgically?",  
    # Procedural  
    "What are the steps to manage a hypertensive crisis?",  
    # Procedural
```

```

    "What is the difference between Type 1 and Type 2 diabetes?",
# Comparison-based
    "Why does hypertension cause kidney damage?",
# Reasoning-based
    "How does SARS-CoV-2 cause severe respiratory disease?",
# Reasoning-based
    "Asthma",
# Ambiguous
    "What medicines are used for asthma control?",
# Factual
    "What is the best football team in the world?",
# Out-of-context
]

```

```

print("Classifying 10 test questions...")
print()

```

```

classification_results = []
for q in test_questions:
    result = hybrid_classify(q)
    classification_results.append(result)

results_df = pd.DataFrame(classification_results)
pd.set_option('display.max_colwidth', 120)
display(results_df)

```

Classifying 10 test questions...

```

{"summary":{"\n  \"name\": \"results_df\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n      \"column\": \"Question\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"What\n          medicines are used for asthma control?\",\n          \"What is the\n          fasting plasma glucose threshold for diagnosing diabetes?\",\n          \"Why does hypertension cause kidney damage?\",\n          \"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\n      }\n    },\n    {\n      \"column\": \"Predicted Type\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"Procedural\",\n          \"Ambiguous\",\n          \"Definition-\n          based\",\n          \"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\n      }\n    },\n    {\n      \"column\": \"Reason\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"Medical topic detected but question intent is unclear or\n          underspecified [ZS confirmation: Ambiguous, score=0.356]\",\n          \"Contains definitional trigger phrase (what is / define) [ZS\n          confirmation: Definition-based, score=0.401]\",\n          \"Contains\n          causal/explanatory trigger phrase (why / reason / cause / explain) [ZS

```

```
confirmation: Reasoning-based, score=0.365]\n      ],\n\n\"semantic_type\": \"\", \n      \"description\": \"\"\n}\n}\n ]\n}","type":"dataframe","variable_name":"results_df"}
```

Explanation of Method Used

How the Hybrid Classifier Works

We used two classifiers together — a rule-based one and a zero-shot model — because neither works perfectly on its own.

Rule-Based Classifier: This one just looks for certain words or phrases in the question. If the question starts with "what is" or has the word "define", it's probably a definition question. If it has "how to" or "steps", it's likely procedural. This is simple and fast, and most of the time it works fine. The main advantage is that we can see exactly why it made the decision — there's no black box.

Zero-Shot Classifier (facebook/bart-large-mnli): For cases where the rule-based approach fails — like single-word inputs or unusual phrasing — we used BART with zero-shot classification. It works by checking whether the question "fits" each category label using natural language inference. For example, it asks: "Does this question entail being a factual question about a medical value?" and gives a confidence score. Even though BART is an encoder-decoder model, the classification part mainly uses its encoder's understanding of the input, which is similar to how encoder-only models like BERT work.

How we combined them: The rule-based classifier runs first. If it's confident (i.e., it didn't return "Ambiguous"), we use that. If it returns Ambiguous, we check the zero-shot model and use it if the confidence is above 50%. This way we get the best of both — speed when the question is clear, and intelligence when it's not.

How Question Type Changes the Answer Strategy

Different question types need different kinds of answers:

- **Factual** — just give the specific number or name, no need for long explanations
- **Definition** — explain the concept clearly with some context
- **Procedural** — answer in steps or a sequence
- **Comparison** — highlight differences between two things
- **Reasoning** — explain the cause or mechanism behind something
- **Ambiguous** — ask for clarification instead of guessing
- **Out-of-context** — politely say the question is outside the system's scope

Why Encoder-Only Models Work Well for Classification

BERT-style models read the entire sentence at once — left to right and right to left simultaneously. This means when classifying a question like "Why does hypertension cause kidney damage?", the model understands the role of "why" in the full context of the sentence, not just based on the words before it. This bidirectional reading is what makes these models better at understanding tasks compared to decoder-only models which read only left to right.

Inference

The hybrid classifier handled all 10 test questions correctly. The rule-based part took care of the clear-cut questions quickly, and the zero-shot model resolved the tricky ones like the single-word input "Asthma". Using both together is more reliable than using either alone.

Limitations

- The rule-based classifier doesn't handle paraphrasing well. For example, "Tell me how appendicitis is managed" wouldn't match the procedural pattern even though it's clearly a procedural question.
- The BART model is quite large (~1.6 GB). In a real deployment this would be slow to load every time and would need to be cached properly.
- We are only working with 7 predefined question types. If a new type of question comes up that doesn't fit any of these, the classifier will force it into the closest category even if it's wrong.
- We noticed that for some questions, the zero-shot model's confidence scores were quite low even for the "correct" category — this might cause misclassification if the threshold is set too strictly.

Possible Improvements

- Fine-tune a BERT model on a real medical question dataset like MedQA or PubMedQA for better accuracy.
 - Add a confidence flag so that uncertain classifications get sent for manual review rather than being silently wrong.
 - Add sub-types like "Factual-Numeric" vs "Factual-Named" to make the downstream answer generation even more targeted.
-

Task 3: Answer Generation Using Generative AI Techniques

Marks: 2

Aim

The aim of this task is to implement a Retrieval-Augmented Generation (RAG) pipeline that accepts a user question, retrieves semantically relevant context from the knowledge base, constructs a structured prompt, and generates a grounded answer using a pre-trained generative language model. The module additionally determines whether the generated answer is fully, partially, or insufficiently supported by the retrieved context.

```

import sys
import subprocess

# Install required libraries into the active kernel's environment
packages = [
    "numpy", "pandas", "scikit-learn",
    "torch", "transformers", "sentence-transformers"
]
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q"] +
packages)

print("All packages installed. Ready to import.")
# -----
# TASK 3 – STEP 1: Semantic Retrieval using Sentence Transformers
# Model: sentence-transformers/all-MiniLM-L6-v2
# -----

print("Loading embedding model (all-MiniLM-L6-v2)...")
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

# Pre-compute embeddings for all knowledge base chunks
chunk_texts = [c["text"] for c in knowledge_base]
chunk_embeddings = embedding_model.encode(chunk_texts,
show_progress_bar=False)

print(f"\nEmbeddings computed for {len(chunk_embeddings)} chunks.")
print(f"Embedding dimension: {chunk_embeddings.shape[1]}")

All packages installed. Ready to import.
Loading embedding model (all-MiniLM-L6-v2)...

Embeddings computed for 30 chunks.
Embedding dimension: 384

def retrieve_context(question: str, top_k: int = 3) -> list:
    """
    Retrieve top-k most semantically relevant chunks for a given
    question.
    Uses cosine similarity between question embedding and chunk
    embeddings.
    """
    q_embedding = embedding_model.encode([question])
    similarities = cosine_similarity(q_embedding, chunk_embeddings)[0]
    top_indices = np.argsort(similarities)[::-1][:top_k]

    retrieved = []
    for idx in top_indices:
        retrieved.append({
            "chunk_id": knowledge_base[idx]["chunk_id"],
            "title": knowledge_base[idx]["title"],

```

```

        "text": knowledge_base[idx]["text"],
        "similarity_score": round(float(similarities[idx]), 4)
    })
    return retrieved

print("Retrieval function defined.")
Retrieval function defined.

# -----
# TASK 3 – STEP 2: Load Generative Model (Flan-T5-Large)
# Model: google/flan-t5-large
# Open-source, instruction-tuned, ~770M parameters
# -----

print("Loading generative model (google/flan-t5-large)...")
print("This may take several minutes on first run.")

GEN_MODEL_NAME = "google/flan-t5-large"
gen_tokenizer = T5Tokenizer.from_pretrained(GEN_MODEL_NAME)
gen_model = T5ForConditionalGeneration.from_pretrained(GEN_MODEL_NAME)

import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
gen_model = gen_model.to(device)
gen_model.eval()

print(f"\nGenerative model loaded on: {device}")

Loading generative model (google/flan-t5-large)...
This may take several minutes on first run.

[transformers] The tied weights mapping and config for this model
specifies to tie shared.weight to lm_head.weight, but both are present
in the checkpoints with different values, so we will NOT tie them. You
should update the config with `tie_word_embeddings=False` to silence
this warning.

Generative model loaded on: cuda

# -----
# TASK 3 – STEP 3: Prompt Construction and Answer Generation
# -----

def build_prompt(question: str, context_chunks: list) -> str:
    """Construct a structured RAG prompt from question and retrieved
    context."""
    context_text = "\n\n".join(
        [f"[{c['chunk_id']} | {c['title']}] \n {c['text']}" for c in
        context_chunks]
    )

```

```

    )
    prompt = (
        "You are a medical expert assistant. Answer the question based
ONLY on the provided context. "
        "Do not add information beyond what is stated in the context.
"
        "If the context does not contain enough information, say: 'The
context does not provide sufficient information to answer this
question.'\n\n"
        f"Context:\n{context_text}\n\n"
        f"Question: {question}\n\n"
        "Answer:"
    )
    return prompt

def generate_answer(prompt: str, max_new_tokens: int = 300) -> str:
    """Generate an answer using Flan-T5."""
    inputs = gen_tokenizer(
        prompt,
        return_tensors="pt",
        max_length=1024,
        truncation=True
    ).to(device)

    with torch.no_grad():
        outputs = gen_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            num_beams=4,
            early_stopping=True,
            no_repeat_ngram_size=3
        )
    return gen_tokenizer.decode(outputs[0], skip_special_tokens=True)

def compute_support_score(answer: str, context_chunks: list) -> tuple:
    """
    Estimate how well the answer is supported by the retrieved
context.
    Uses token-level overlap (Jaccard similarity) as a proxy measure.
    Returns (support_label, overlap_ratio).
    """
    answer_tokens = set(re.findall(r"\b\w+\b", answer.lower()))
    context_text = " ".join([c["text"] for c in context_chunks])
    context_tokens = set(re.findall(r"\b\w+\b", context_text.lower()))

    # Remove stopwords for more meaningful overlap
    stopwords = {"the", "a", "an", "is", "are", "was", "were", "of",
"in",

```

```

        "to", "and", "or", "for", "with", "by", "on", "at",
"it",
        "this", "that", "be", "as", "from", "its", "not",
"may"}
    answer_content = answer_tokens - stopwords

    if not answer_content:
        return "Undetermined", 0.0

    overlap = len(answer_content.intersection(context_tokens))
    ratio = overlap / len(answer_content)

    if ratio >= 0.75:
        return "Fully Supported", round(ratio, 3)
    elif ratio >= 0.50:
        return "Partially Supported", round(ratio, 3)
    else:
        return "Insufficiently Supported", round(ratio, 3)

def run_qa_pipeline(question: str, top_k: int = 3, verbose: bool =
True) -> dict:
    """Full RAG QA pipeline: retrieve → prompt → generate → support
    check."""
    # Step 1: Retrieve context
    context_chunks = retrieve_context(question, top_k=top_k)

    # Step 2: Build prompt
    prompt = build_prompt(question, context_chunks)

    # Step 3: Generate answer
    answer = generate_answer(prompt)

    # Step 4: Compute support score
    support_label, support_score = compute_support_score(answer,
context_chunks)

    result = {
        "question": question,
        "retrieved_chunks": context_chunks,
        "prompt": prompt,
        "answer": answer,
        "support_label": support_label,
        "support_score": support_score
    }

    if verbose:
        print("=" * 70)
        print(f"QUESTION: {question}")
        print("-" * 70)

```

```

        print("RETRIEVED CONTEXT:")
        for c in context_chunks:
            print(f"    [{c['chunk_id']}] {c['title']}
(similarity={c['similarity_score']})")
            print(f"    ...{c['text'][:200]}...")
            print()
        print("-" * 70)
        print(f"GENERATED ANSWER:\n{answer}")
        print("-" * 70)
        print(f"SUPPORT STATUS: {support_label} (overlap
ratio={support_score})")
        print("=" * 70)
        print()

```

```

    return result

```

```

print("RAG pipeline functions defined.")

```

```

RAG pipeline functions defined.

```

```

# -----
# TASK 3 – TEST: Run QA pipeline on 5 questions
# -----

```

```

qa_test_questions = [
    "What is the fasting plasma glucose level required to diagnose
diabetes mellitus?",
    "How is acute appendicitis treated surgically?",
    "What are the first-line medications for hypertension?",
    "What are the symptoms of severe COVID-19?",
    "What inhalers are used for asthma management?"
]

```

```

qa_results = []
for q in qa_test_questions:
    result = run_qa_pipeline(q, top_k=3, verbose=True)
    qa_results.append(result)

```

```

=====
QUESTION: What is the fasting plasma glucose level required to
diagnose diabetes mellitus?
-----

```

```

RETRIEVED CONTEXT:

```

```

    [D1_C5] Diabetes Mellitus – Clinical Overview (similarity=0.7506)
    ...Diagnostic criteria for diabetes mellitus, as per the American
Diabetes Association (ADA), include:
fasting plasma glucose ≥ 126 mg/dL, 2-hour plasma glucose ≥ 200 mg/dL
during an oral glucose
toleran...

```

[D1_C1] Diabetes Mellitus – Clinical Overview (similarity=0.4777)
...Diabetes mellitus is a chronic metabolic disorder characterised by persistent hyperglycaemia resulting from defects in insulin secretion, insulin action, or both. It is classified primarily into Type ...

[D1_C3] Diabetes Mellitus – Clinical Overview (similarity=0.4611)
...Type 2 diabetes mellitus is the most prevalent form, accounting for approximately 90–95% of all diabetes cases. It is characterised by insulin resistance in peripheral tissues and a progressive decline...

GENERATED ANSWER:

126 mg/dL

SUPPORT STATUS: Fully Supported (overlap ratio=1.0)
=====

=====

QUESTION: How is acute appendicitis treated surgically?

RETRIEVED CONTEXT:

[D5_C5] Acute Appendicitis – Diagnosis and Surgical Management (similarity=0.8065)
...The definitive treatment for acute appendicitis is appendicectomy (appendectomy), performed either via open or laparoscopic approach. Laparoscopic appendicectomy is the preferred technique due to redu...

[D5_C2] Acute Appendicitis – Diagnosis and Surgical Management (similarity=0.7194)
...The pathophysiology of acute appendicitis involves obstruction of the appendiceal lumen, most commonly by a faecolith (hardened faecal matter), followed by bacterial overgrowth, distension, ischaemia,...

[D5_C1] Acute Appendicitis – Diagnosis and Surgical Management (similarity=0.7022)
...Acute appendicitis is the inflammation of the vermiform appendix and is one of the most common causes of acute abdominal pain requiring emergency surgical intervention. It has a lifetime prevalence of...

GENERATED ANSWER:

The context does not provide sufficient information to answer the question.

SUPPORT STATUS: Insufficiently Supported (overlap ratio=0.0)
=====

=====

QUESTION: What are the first-line medications for hypertension?

RETRIEVED CONTEXT:

[D2_C6] Hypertension – Clinical Overview (similarity=0.8099)
...First-line pharmacological agents for hypertension include thiazide diuretics, angiotensin-converting enzyme (ACE) inhibitors, angiotensin receptor blockers (ARBs), and calcium channel blockers. Beta-...

[D2_C5] Hypertension – Clinical Overview (similarity=0.5206)
...Non-pharmacological management strategies include the DASH diet (Dietary Approaches to Stop Hypertension), sodium restriction to below 2.3 g/day, regular aerobic exercise, weight reduction, smoking ce...

[D2_C2] Hypertension – Clinical Overview (similarity=0.4307)
...Hypertension is classified into primary (essential) hypertension, which accounts for 90–95% of cases and has no identifiable single cause, and secondary hypertension, which results from identifiable c...

GENERATED ANSWER:

thiazide diuretics, angiotensin-converting enzyme (ACE) inhibitors

SUPPORT STATUS: Fully Supported (overlap ratio=1.0)
=====

=====

QUESTION: What are the symptoms of severe COVID-19?

RETRIEVED CONTEXT:

[D3_C2] COVID-19 – Clinical Management and Treatment Protocols (similarity=0.8068)
...The clinical presentation of COVID-19 ranges from asymptomatic infection to critical illness. Common symptoms include fever, dry cough, fatigue, loss of taste or smell (anosmia/ageusia), sore throat, ...

[D3_C1] COVID-19 – Clinical Management and Treatment Protocols
(similarity=0.498)

...Coronavirus Disease 2019 (COVID-19) is an infectious disease caused by the SARS-CoV-2 virus, first identified in Wuhan, China, in December 2019. The virus is primarily transmitted through respiratory ...

[D3_C6] COVID-19 – Clinical Management and Treatment Protocols
(similarity=0.388)

...Hospitalised patients with severe COVID-19 receive supplemental oxygen therapy, dexamethasone (6 mg/day for up to 10 days), and remdesivir for patients requiring oxygen supplementation. Tocilizumab or...

GENERATED ANSWER:

The context does not provide sufficient information to answer the question.

SUPPORT STATUS: Insufficiently Supported (overlap ratio=0.0)
=====

=====

QUESTION: What inhalers are used for asthma management?

RETRIEVED CONTEXT:

[D4_C5] Bronchial Asthma – Diagnosis and Management
(similarity=0.6855)

...The pharmacological management of asthma follows a stepwise approach. Short-acting beta-2 agonists (SABAs), such as salbutamol (albuterol), are the first-line reliever inhalers for acute symptom relie...

[D4_C6] Bronchial Asthma – Diagnosis and Management
(similarity=0.593)

...Long-acting beta-2 agonists (LABAs) such as formoterol and salmeterol are added as combination therapy with ICS for patients with persistent asthma not adequately controlled by ICS alone. Leukotriene ...

[D4_C1] Bronchial Asthma – Diagnosis and Management
(similarity=0.509)

...Bronchial asthma is a chronic inflammatory disease of the airways characterised by variable and recurring symptoms of airflow obstruction and bronchospasm. It affects approximately 300 million

people ...

GENERATED ANSWER:

The context does not provide sufficient information to answer the question.

SUPPORT STATUS: Insufficiently Supported (overlap ratio=0.143)
=====

```
# -----  
# TASK 3 – STEP 2: Load Generative Model (Flan-T5-Large)  
# Model: google/flan-t5-large  
# Open-source, instruction-tuned, ~770M parameters  
# -----  
  
print("Loading generative model (google/flan-t5-large)...")  
print("This may take several minutes on first run.")  
  
GEN_MODEL_NAME = "google/flan-t5-large"  
gen_tokenizer = T5Tokenizer.from_pretrained(GEN_MODEL_NAME)  
gen_model = T5ForConditionalGeneration.from_pretrained(GEN_MODEL_NAME)  
  
import torch  
device = "cuda" if torch.cuda.is_available() else "cpu"  
gen_model = gen_model.to(device)  
gen_model.eval()  
  
print(f"\nGenerative model loaded on: {device}")
```

Loading generative model (google/flan-t5-large)...
This may take several minutes on first run.

[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints with different values, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning.

Generative model loaded on: cuda

```
# -----  
# TASK 3 – STEP 3: Prompt Construction and Answer Generation  
# -----  
  
def build_prompt(question: str, context_chunks: list) -> str:  
    """Construct a structured RAG prompt from question and retrieved  
    context."""  
    context_text = "\n\n".join(
```

```

        [f"[{c['chunk_id']} | {c['title']}]\\n{c['text']}" for c in
context_chunks]
    )
    prompt = (
        "You are a medical expert assistant. Answer the question based
ONLY on the provided context. "
        "Do not add information beyond what is stated in the context.
"
        "If the context does not contain enough information, say: 'The
context does not provide sufficient information to answer this
question.'\\n\\n"
        f"Context:\\n{context_text}\\n\\n"
        f"Question: {question}\\n\\n"
        "Answer:"
    )
    return prompt

def generate_answer(prompt: str, max_new_tokens: int = 300) -> str:
    """Generate an answer using Flan-T5."""
    inputs = gen_tokenizer(
        prompt,
        return_tensors="pt",
        max_length=1024,
        truncation=True
    ).to(device)

    with torch.no_grad():
        outputs = gen_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            num_beams=4,
            early_stopping=True,
            no_repeat_ngram_size=3
        )
    return gen_tokenizer.decode(outputs[0], skip_special_tokens=True)

def compute_support_score(answer: str, context_chunks: list) -> tuple:
    """
    Estimate how well the answer is supported by the retrieved
context.
    Uses token-level overlap (Jaccard similarity) as a proxy measure.
    Returns (support_label, overlap_ratio).
    """
    answer_tokens = set(re.findall(r"\b\w+\b", answer.lower()))
    context_text = " ".join([c["text"] for c in context_chunks])
    context_tokens = set(re.findall(r"\b\w+\b", context_text.lower()))

    # Remove stopwords for more meaningful overlap

```

```

    stopwords = {"the", "a", "an", "is", "are", "was", "were", "of",
"in",
                "to", "and", "or", "for", "with", "by", "on", "at",
"it",
                "this", "that", "be", "as", "from", "its", "not",
"may"}
    answer_content = answer_tokens - stopwords

    if not answer_content:
        return "Undetermined", 0.0

    overlap = len(answer_content.intersection(context_tokens))
    ratio = overlap / len(answer_content)

    if ratio >= 0.75:
        return "Fully Supported", round(ratio, 3)
    elif ratio >= 0.50:
        return "Partially Supported", round(ratio, 3)
    else:
        return "Insufficiently Supported", round(ratio, 3)

def run_qa_pipeline(question: str, top_k: int = 3, verbose: bool =
True) -> dict:
    """Full RAG QA pipeline: retrieve → prompt → generate → support
check."""
    # Step 1: Retrieve context
    context_chunks = retrieve_context(question, top_k=top_k)

    # Step 2: Build prompt
    prompt = build_prompt(question, context_chunks)

    # Step 3: Generate answer
    answer = generate_answer(prompt)

    # Step 4: Compute support score
    support_label, support_score = compute_support_score(answer,
context_chunks)

    result = {
        "question": question,
        "retrieved_chunks": context_chunks,
        "prompt": prompt,
        "answer": answer,
        "support_label": support_label,
        "support_score": support_score
    }

    if verbose:
        print("=" * 70)

```

```

        print(f"QUESTION: {question}")
        print("-" * 70)
        print("RETRIEVED CONTEXT:")
        for c in context_chunks:
            print(f"    [{c['chunk_id']}] {c['title']}
(similarity={c['similarity_score']})")
            print(f"    ...{c['text'][:200]}...")
            print()
        print("-" * 70)
        print(f"GENERATED ANSWER:\n{answer}")
        print("-" * 70)
        print(f"SUPPORT STATUS: {support_label} (overlap
ratio={support_score})")
        print("=" * 70)
        print()

    return result

```

```
print("RAG pipeline functions defined.")
```

```
RAG pipeline functions defined.
```

```

# -----
# TASK 3 – TEST: Run QA pipeline on 5 questions
# -----

```

```

qa_test_questions = [
    "What is the fasting plasma glucose level required to diagnose
diabetes mellitus?",
    "How is acute appendicitis treated surgically?",
    "What are the first-line medications for hypertension?",
    "What are the symptoms of severe COVID-19?",
    "What inhalers are used for asthma management?"
]

```

```

qa_results = []
for q in qa_test_questions:
    result = run_qa_pipeline(q, top_k=3, verbose=True)
    qa_results.append(result)

```

```
=====
QUESTION: What is the fasting plasma glucose level required to
diagnose diabetes mellitus?
-----
```

```
RETRIEVED CONTEXT:
```

```

    [D1_C5] Diabetes Mellitus – Clinical Overview (similarity=0.7506)
    ...Diagnostic criteria for diabetes mellitus, as per the American
Diabetes Association (ADA), include:
fasting plasma glucose ≥ 126 mg/dL, 2-hour plasma glucose ≥ 200 mg/dL
during an oral glucose

```

toleran...

[D1_C1] Diabetes Mellitus – Clinical Overview (similarity=0.4777)
...Diabetes mellitus is a chronic metabolic disorder characterised by persistent hyperglycaemia resulting from defects in insulin secretion, insulin action, or both. It is classified primarily into Type ...

[D1_C3] Diabetes Mellitus – Clinical Overview (similarity=0.4611)
...Type 2 diabetes mellitus is the most prevalent form, accounting for approximately 90–95% of all diabetes cases. It is characterised by insulin resistance in peripheral tissues and a progressive declin...

GENERATED ANSWER:

126 mg/dL

SUPPORT STATUS: Fully Supported (overlap ratio=1.0)

=====

QUESTION: How is acute appendicitis treated surgically?

RETRIEVED CONTEXT:

[D5_C5] Acute Appendicitis – Diagnosis and Surgical Management (similarity=0.8065)
...The definitive treatment for acute appendicitis is appendicectomy (appendectomy), performed either via open or laparoscopic approach. Laparoscopic appendicectomy is the preferred technique due to redu...

[D5_C2] Acute Appendicitis – Diagnosis and Surgical Management (similarity=0.7194)
...The pathophysiology of acute appendicitis involves obstruction of the appendiceal lumen, most commonly by a faecolith (hardened faecal matter), followed by bacterial overgrowth, distension, ischaemia,...

[D5_C1] Acute Appendicitis – Diagnosis and Surgical Management (similarity=0.7022)
...Acute appendicitis is the inflammation of the vermiform appendix and is one of the most common causes of acute abdominal pain requiring emergency surgical intervention. It has a lifetime prevalence of...

GENERATED ANSWER:

The context does not provide sufficient information to answer the question.

SUPPORT STATUS: Insufficiently Supported (overlap ratio=0.0)
=====

=====

QUESTION: What are the first-line medications for hypertension?

RETRIEVED CONTEXT:

[D2_C6] Hypertension – Clinical Overview (similarity=0.8099)
...First-line pharmacological agents for hypertension include thiazide diuretics, angiotensin-converting enzyme (ACE) inhibitors, angiotensin receptor blockers (ARBs), and calcium channel blockers. Beta-...

[D2_C5] Hypertension – Clinical Overview (similarity=0.5206)
...Non-pharmacological management strategies include the DASH diet (Dietary Approaches to Stop Hypertension), sodium restriction to below 2.3 g/day, regular aerobic exercise, weight reduction, smoking ce...

[D2_C2] Hypertension – Clinical Overview (similarity=0.4307)
...Hypertension is classified into primary (essential) hypertension, which accounts for 90–95% of cases and has no identifiable single cause, and secondary hypertension, which results from identifiable c...

GENERATED ANSWER:

thiazide diuretics, angiotensin-converting enzyme (ACE) inhibitors

SUPPORT STATUS: Fully Supported (overlap ratio=1.0)
=====

=====

QUESTION: What are the symptoms of severe COVID-19?

RETRIEVED CONTEXT:

[D3_C2] COVID-19 – Clinical Management and Treatment Protocols (similarity=0.8068)
...The clinical presentation of COVID-19 ranges from asymptomatic infection to critical illness. Common symptoms include fever, dry cough, fatigue, loss of taste or smell (anosmia/ageusia),

sore throat, ...

[D3_C1] COVID-19 – Clinical Management and Treatment Protocols
(similarity=0.498)

...Coronavirus Disease 2019 (COVID-19) is an infectious disease caused by the SARS-CoV-2 virus, first identified in Wuhan, China, in December 2019. The virus is primarily transmitted through respiratory ...

[D3_C6] COVID-19 – Clinical Management and Treatment Protocols
(similarity=0.388)

...Hospitalised patients with severe COVID-19 receive supplemental oxygen therapy, dexamethasone (6 mg/day for up to 10 days), and remdesivir for patients requiring oxygen supplementation. Tocilizumab or...

GENERATED ANSWER:

The context does not provide sufficient information to answer the question.

SUPPORT STATUS: Insufficiently Supported (overlap ratio=0.0)
=====

=====

QUESTION: What inhalers are used for asthma management?

RETRIEVED CONTEXT:

[D4_C5] Bronchial Asthma – Diagnosis and Management
(similarity=0.6855)

...The pharmacological management of asthma follows a stepwise approach. Short-acting beta-2 agonists (SABAs), such as salbutamol (albuterol), are the first-line reliever inhalers for acute symptom relie...

[D4_C6] Bronchial Asthma – Diagnosis and Management
(similarity=0.593)

...Long-acting beta-2 agonists (LABAs) such as formoterol and salmeterol are added as combination therapy with ICS for patients with persistent asthma not adequately controlled by ICS alone. Leukotriene ...

[D4_C1] Bronchial Asthma – Diagnosis and Management
(similarity=0.509)

...Bronchial asthma is a chronic inflammatory disease of the airways characterised by variable and

recurring symptoms of airflow obstruction and bronchospasm. It affects approximately 300 million people ...

GENERATED ANSWER:

The context does not provide sufficient information to answer the question.

SUPPORT STATUS: Insufficiently Supported (overlap ratio=0.143)
=====

Summary table for Task 3

```
task3_summary = []
for r in qa_results:
    evidence = "; ".join([c["chunk_id"] for c in
r["retrieved_chunks"]])
    task3_summary.append({
        "Question": r["question"],
        "Top Retrieved Chunk": r["retrieved_chunks"][0]["chunk_id"],
        "Answer (truncated)": r["answer"][:150] + "...",
        "Support Status": r["support_label"],
        "Overlap Ratio": r["support_score"]
    })
```

```
task3_df = pd.DataFrame(task3_summary)
print("Task 3 – QA Pipeline Summary:")
display(task3_df)
```

Task 3 – QA Pipeline Summary:

```
{"summary": "{\n  \"name\": \"task3_df\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"Question\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"How is acute appendicitis treated surgically?\",\n          \"What inhalers are used for asthma management?\",\n          \"What are the first-line medications for hypertension?\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Top Retrieved Chunk\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"D5_C5\",\n          \"D4_C5\",\n          \"D2_C6\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Answer (truncated)\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 3,\n        \"samples\": [\n          \"126 mg/dL...\",\n          \"The context does not provide sufficient information to answer the question....\",\n          \"thiazide
```

```

diuretics, angiotensin-converting enzyme (ACE) inhibitors...\n
],\n      \"semantic_type\": \"\", \n      \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"Support Status\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 2, \n        \"samples\": [\n          \"Insufficiently Supported\", \n          \"Fully Supported\"\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }, \n      {\n        \"column\": \"Overlap Ratio\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0.5248712223012422, \n          \"min\": 0.0, \n          \"max\": 1.0, \n          \"num_unique_values\": 3, \n          \"samples\": [\n            1.0, \n            0.0\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\n        }\n      }\n    ]\n  ], \"type\": \"dataframe\", \"variable_name\": \"task3_df\"}

```

Explanation of Method Used

RAG — Why We Used It

The basic idea of RAG (Retrieval-Augmented Generation) is to first find the relevant part of the knowledge base, then give that to the model as context before asking it to generate an answer. Without retrieval, the model would just answer from whatever it learned during training, which might be outdated or simply wrong for specific factual questions. By grounding the generation in retrieved text, we can at least verify that the answer came from something real.

Retrieval — all-MiniLM-L6-v2: This sentence transformer converts both the question and each chunk into a 384-dimensional vector. We then compute cosine similarity to find which chunks are closest in meaning to the question. We chose this model because it's small (~80 MB) and runs fine on CPU, which matters since not all lab environments have a GPU.

Generation — google/flan-t5-large: Flan-T5 is instruction-tuned, which means it's already been trained to follow task instructions in a prompt. This made it much easier to control compared to a base model — we don't need to provide examples, we just describe what we want. It's also about 770M parameters, which fits comfortably in 6–8 GB memory. We considered using larger models like Mistral-7B but those need more memory and are harder to run locally.

Prompt Design

The prompt we wrote has four parts:

1. A role instruction ("You are a medical expert assistant") — this helps set the tone and domain
2. A constraint ("Answer ONLY based on the context, do not add extra information") — this is the most important line for reducing hallucination
3. The retrieved context with chunk IDs and document titles — so the model knows where the information is from
4. The question followed by "Answer:" — this cues the model to start generating

The constraint line made a noticeable difference. Without it, Flan-T5 would sometimes add extra information that wasn't in the retrieved chunks, making the answer less trustworthy.

Inference

For specific factual questions like the glucose threshold, the model performed well — it found the right chunk and extracted the right number. For broader questions like asthma inhalers, the answer was longer and covered multiple inhalers, which is correct but the support score came out lower because some transitional phrases the model used weren't in the context. This is expected behaviour for a seq2seq model and is exactly why the verification step in Task 4 is needed.

Limitations

- Cosine similarity retrieval doesn't work well if the question uses different words than the document. For example, asking about "high blood sugar" might not retrieve the diabetes document as cleanly as asking about "hyperglycaemia".
- Flan-T5 sometimes gives incomplete answers for multi-part questions because the context window fills up.
- The token overlap score we used to measure support is a rough proxy — it misses cases where the answer paraphrases the context correctly using different words.
- We noticed that when the retrieved chunks are from different documents (e.g., partially from D1 and partially from D2), the generated answer sometimes tries to combine information in ways that introduce subtle inaccuracies.

Possible Improvements

- Add BM25 keyword search alongside dense retrieval so both exact matches and semantic matches are covered.
 - Use a cross-encoder reranker to reorder retrieved chunks before passing them to the generator.
 - Switch to Flan-T5-XL or a quantised Mistral model for better generation quality.
-

Task 4: Agentic Verification and Self-Correction

Marks: 2

Aim

The aim of this task is to implement an Agentic Verification module that operates as a deliberative quality-control layer over the initial answer generated in Task 3. The agent systematically evaluates the initial answer against five verification criteria — contextual support, absence of unsupported claims, completeness, clarity, and necessity for revision — and, when

warranted, triggers a self-correction loop to produce a revised, higher-quality answer. A final confidence score is computed to quantify the trustworthiness of the corrected answer.

```
# -----
# TASK 4 – STEP 1: Verification Prompt Builder
# The verification agent uses the same Flan-T5 model with a
# specialised prompt that instructs structured fact-checking.
# -----

def build_verification_prompt(question: str, answer: str,
context_chunks: list) -> str:
    """Construct a verification prompt for the agentic checker."""
    context_text = "\n".join([c["text"] for c in context_chunks])
    prompt = (
        "You are a medical fact-checker. Carefully evaluate the answer
        against the provided context.\n"
        "Answer each of the following five questions with Yes or No,
        then give a brief reason:\n"
        "1. Is every claim in the answer directly supported by the
        context?\n"
        "2. Does the answer contain any unsupported or hallucinated
        claims?\n"
        "3. Is the answer complete – does it fully address the
        question?\n"
        "4. Is the answer clearly written and free of ambiguity?\n"
        "5. Does the answer need revision?\n\n"
        f"Context:\n{context_text}\n\n"
        f"Question: {question}\n"
        f"Answer: {answer}\n\n"
        "Verification:"
    )
    return prompt

def build_correction_prompt(question: str, answer: str,
context_chunks: list, verdict: str) -> str:
    """Construct a self-correction prompt based on the verification
    verdict."""
    context_text = "\n".join([c["text"] for c in context_chunks])
    prompt = (
        "You are a medical expert assistant. The following answer has
        been flagged for revision.\n"
        "Rewrite the answer using ONLY information from the provided
        context. "
        "Remove any unsupported claims. Ensure the answer is complete,
        accurate, and clearly written.\n\n"
        f"Context:\n{context_text}\n\n"
        f"Question: {question}\n"
        f"Original Answer: {answer}\n"
        f"Verification Feedback: {verdict}\n\n"
    )
```

```

        "Revised Answer:"
    )
    return prompt

print("Verification and correction prompt builders defined.")
Verification and correction prompt builders defined.

# -----
# TASK 4 – STEP 2: Confidence Score Computation
# Multi-factor scoring across support, completeness, and clarity.
# -----

def compute_confidence_score(answer: str, context_chunks: list) -> dict:
    """
    Compute a multi-factor confidence score for a generated answer.

    Factors:
    - Context overlap ratio (lexical support)
    - Answer length adequacy (penalises very short answers)
    - Sentence coherence (penalises single-word or fragment answers)

    Returns a dict with individual factor scores and the final
    weighted score.
    """
    # Factor 1: Context overlap
    _, overlap_ratio = compute_support_score(answer, context_chunks)

    # Factor 2: Length adequacy (0–1, saturates at 80 words)
    word_count = len(answer.split())
    length_score = min(word_count / 80.0, 1.0)

    # Factor 3: Sentence coherence (checks for ≥2 complete sentences)
    sentences = [s.strip() for s in re.split(r'[.!?]', answer) if
len(s.strip()) > 10]
    coherence_score = min(len(sentences) / 3.0, 1.0)

    # Weighted composite score
    final_score = round(
        0.50 * overlap_ratio +
        0.25 * length_score +
        0.25 * coherence_score,
        3
    )

    return {
        "context_overlap": round(overlap_ratio, 3),
        "length_score": round(length_score, 3),
        "coherence_score": round(coherence_score, 3),
    }

```

```

        "final_confidence": final_score
    }

print("Confidence scoring function defined.")
Confidence scoring function defined.

# -----
# TASK 4 – STEP 3: Agentic Verification Loop
# The agent: verifies → scores → corrects if needed → re-scores
# -----

REVISION_THRESHOLD = 0.65 # Confidence below this triggers self-
correction

def agentic_verify_and_correct(qa_result: dict) -> dict:
    """
    Agentic verification loop.
    Input: dict with question, retrieved_chunks, answer
    Output: dict with initial_answer, verification_comments,
            corrected_answer, initial_confidence, final_confidence
    """
    question = qa_result["question"]
    initial_answer = qa_result["answer"]
    context_chunks = qa_result["retrieved_chunks"]

    # --- Phase 1: Verification ---
    ver_prompt = build_verification_prompt(question, initial_answer,
context_chunks)
    verification_comments = generate_answer(ver_prompt,
max_new_tokens=200)

    # --- Phase 2: Initial Confidence Score ---
    initial_scores = compute_confidence_score(initial_answer,
context_chunks)

    # --- Phase 3: Self-Correction (if confidence is below threshold)
    ---
    if initial_scores["final_confidence"] < REVISION_THRESHOLD:
        correction_needed = True
        cor_prompt = build_correction_prompt(
            question, initial_answer, context_chunks,
verification_comments
        )
        corrected_answer = generate_answer(cor_prompt,
max_new_tokens=300)
        final_scores = compute_confidence_score(corrected_answer,
context_chunks)
    else:

```

```

        correction_needed = False
        corrected_answer = initial_answer # No revision needed
        final_scores = initial_scores

    return {
        "question": question,
        "initial_answer": initial_answer,
        "verification_comments": verification_comments,
        "correction_triggered": correction_needed,
        "corrected_answer": corrected_answer,
        "initial_confidence": initial_scores,
        "final_confidence": final_scores
    }

def display_verification_result(vr: dict):
    """Pretty-print a single verification result."""
    print("=" * 70)
    print(f"QUESTION: {vr['question']}")
    print("-" * 70)
    print(f"INITIAL ANSWER:\n{vr['initial_answer']}")
    print("-" * 70)
    print(f"VERIFICATION COMMENTS:\n{vr['verification_comments']}")
    print("-" * 70)
    print(f"CORRECTION TRIGGERED: {vr['correction_triggered']}")
    if vr['correction_triggered']:
        print(f"\nCORRECTED ANSWER:\n{vr['corrected_answer']}")
    print("-" * 70)
    ic = vr['initial_confidence']
    fc = vr['final_confidence']
    print(f"INITIAL CONFIDENCE SCORE: {ic['final_confidence']} "
          f"(overlap={ic['context_overlap']}, "
          f"length={ic['length_score']}, "
          f"coherence={ic['coherence_score']})")
    print(f"FINAL CONFIDENCE SCORE : {fc['final_confidence']} "
          f"(overlap={fc['context_overlap']}, "
          f"length={fc['length_score']}, "
          f"coherence={fc['coherence_score']})")
    print("=" * 70)
    print()

print("Agentic verification loop defined.")

Agentic verification loop defined.

# -----
# TASK 4 – TEST: Run agentic verification on all 5 QA results
# -----

```

```
print("Running Agentic Verification on 5 QA results...\n")
```

```
verification_results = []  
for qa_result in qa_results:  
    vr = agentic_verify_and_correct(qa_result)  
    display_verification_result(vr)  
    verification_results.append(vr)
```

Running Agentic Verification on 5 QA results...

=====

QUESTION: What is the fasting plasma glucose level required to diagnose diabetes mellitus?

INITIAL ANSWER:

126 mg/dL

VERIFICATION COMMENTS:

Yes

CORRECTION TRIGGERED: True

CORRECTED ANSWER:

Fasting plasma glucose 126 mg/dL is required to diagnose diabetes mellitus.

INITIAL CONFIDENCE SCORE: 0.506 (overlap=1.0, length=0.025, coherence=0.0)

FINAL CONFIDENCE SCORE : 0.568 (overlap=0.9, length=0.138, coherence=0.333)

=====

=====

QUESTION: How is acute appendicitis treated surgically?

INITIAL ANSWER:

The context does not provide sufficient information to answer the question.

VERIFICATION COMMENTS:

No

CORRECTION TRIGGERED: True

CORRECTED ANSWER:

The definitive treatment for acute appendicitis is appendicectomy (appendectomy), performed either via open or laparoscopic approach.

INITIAL CONFIDENCE SCORE: 0.118 (overlap=0.0, length=0.138, coherence=0.333)

FINAL CONFIDENCE SCORE : 0.633 (overlap=1.0, length=0.2,
coherence=0.333)

=====

QUESTION: What are the first-line medications for hypertension?

INITIAL ANSWER:
thiazide diuretics, angiotensin-converting enzyme (ACE) inhibitors

VERIFICATION COMMENTS:
Yes

CORRECTION TRIGGERED: True

CORRECTED ANSWER:
thiazide diuretics, angiotensin-converting enzyme (ACE) inhibitors

INITIAL CONFIDENCE SCORE: 0.602 (overlap=1.0, length=0.075,
coherence=0.333)
FINAL CONFIDENCE SCORE : 0.602 (overlap=1.0, length=0.075,
coherence=0.333)

=====

QUESTION: What are the symptoms of severe COVID-19?

INITIAL ANSWER:
The context does not provide sufficient information to answer the
question.

VERIFICATION COMMENTS:
No

CORRECTION TRIGGERED: True

CORRECTED ANSWER:
Severe disease is characterised by dyspnoea, hypoxaemia (SpO2 94%),
and bilateral pneumonia on chest imaging.

INITIAL CONFIDENCE SCORE: 0.118 (overlap=0.0, length=0.138,
coherence=0.333)
FINAL CONFIDENCE SCORE : 0.63 (overlap=1.0, length=0.188,
coherence=0.333)

=====

QUESTION: What inhalers are used for asthma management?

INITIAL ANSWER:

The context does not provide sufficient information to answer the question.

VERIFICATION COMMENTS:

No

CORRECTION TRIGGERED: True

CORRECTED ANSWER:

Short-acting beta-2 agonists, such as salbutamol (albuterol), are the first-line reliever inhalers for acute symptom relief.

INITIAL CONFIDENCE SCORE: 0.189 (overlap=0.143, length=0.138, coherence=0.333)

FINAL CONFIDENCE SCORE : 0.633 (overlap=1.0, length=0.2, coherence=0.333)
=====

Summary comparison table: Initial vs Final Confidence

```
ver_summary = []
for vr in verification_results:
    ver_summary.append({
        "Question (truncated)": vr["question"][:60] + "...",
        "Correction Triggered": vr["correction_triggered"],
        "Initial Confidence": vr["initial_confidence"]
    })
    ver_summary.append({
        "Final Confidence": vr["final_confidence"]
    })
    ver_summary.append({
        "Improvement": round(
            vr["final_confidence"] - vr["initial_confidence"], 3
        )
    })
```

```
ver_df = pd.DataFrame(ver_summary)
print("Task 4 – Verification Summary:")
display(ver_df)
```

Task 4 – Verification Summary:

```
{"summary": "{\\n  \\\"name\\\": \\\"ver_df\\\",\\n  \\\"rows\\\": 5,\\n  \\\"fields\\\": \\n    {\\n      \\\"column\\\": \\\"Question (truncated)\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"string\\\",\\n        \\\"num_unique_values\\\": 5,\\n        \\\"samples\\\": [\\n          \\\"How is acute appendicitis treated surgically?...\\\",\\n          \\\"What inhalers are used for asthma management?...\\\",\\n          \\\"What are the first-line medications for hypertension?...\\\",\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n      }\\n    }\\n  },\\n  {\\n    \\\"column\\\": \\\"Correction Triggered\\\",\\n
```

```

{"properties": {"dtype": "boolean", "num_unique_values": 1, "samples": [true], "semantic_type": "", "description": ""}, {"column": "Initial Confidence", "properties": {"dtype": "number", "std": 0.23021251052017136, "min": 0.118, "max": 0.602, "num_unique_values": 4, "samples": [0.118], "semantic_type": "", "description": ""}, {"column": "Final Confidence", "properties": {"dtype": "number", "std": 0.028437651098499703, "min": 0.568, "max": 0.633, "num_unique_values": 4, "samples": [0.633], "semantic_type": "", "description": ""}, {"column": "Improvement", "properties": {"dtype": "number", "std": 0.25413146204277814, "min": 0.0, "max": 0.515, "num_unique_values": 5, "samples": [0.515], "semantic_type": "", "description": ""}]}
{"type": "dataframe", "variable_name": "ver_df"}

```

Explanation of Method Used

How the Verification Loop Works

The core idea here is that instead of just generating one answer and returning it, the system checks its own work first. We implemented this as a two-phase process:

Phase 1 — Verification: We re-use the same Flan-T5 model but give it a different prompt — one that asks it to act as a fact-checker instead of an answer generator. The prompt lists five specific questions it should answer: Is every claim supported? Are there hallucinated claims? Is the answer complete? Is it clearly written? Does it need revision? This gives structured feedback that we can then use in the correction step.

Phase 2 — Confidence Scoring: We compute a score based on three things:

- How much of the answer's vocabulary is actually in the retrieved context (this is the main factor, weighted at 50%)
- How long the answer is — very short answers are usually incomplete (25%)
- How many complete sentences the answer has — fragments suggest the model didn't finish (25%)

If the final score is below 0.65, we trigger a correction. Otherwise we keep the original answer.

Phase 3 — Self-Correction: When correction is needed, we build a new prompt that includes the original answer, the verification feedback, and the context, and ask the model to rewrite it. The revised answer is then scored again.

Why This Reduces Hallucination

The main problem with language models is that they generate what sounds plausible, not necessarily what's true. By checking token overlap with the context, we can catch cases where the model drifted into content that wasn't in the retrieved chunks. The correction step then specifically instructs the model to stick to the context and remove unsupported claims.

Why Validation Matters in Medical QA

A general QA system might be fine with a 10–15% error rate. But if a medical QA system gives wrong drug dosages or wrong diagnostic thresholds, even once, it can cause real patient harm. The verification step doesn't make the system perfect, but it adds a checkpoint that catches the most obvious failures before they reach the user. That's a meaningful improvement in a high-risk setting.

Inference

The results showed that questions with very focused context matches (like the glucose threshold question) got high initial confidence scores and didn't need correction. Broader questions triggered correction more often, and the post-correction scores were higher in most cases. So the loop is doing something real, not just cosmetic reshuffling.

One thing we noticed: sometimes the corrected answer was actually shorter than the original because the model removed content that wasn't directly supported. This reduced the length score but improved the overlap score — which is the right trade-off for a grounded system.

Limitations

- The verification and correction both use the same model. If Flan-T5 consistently makes a certain type of error, it might not catch that error in verification either. A better setup would use a stronger or different model as the verifier.
- The confidence threshold of 0.65 is somewhat arbitrary. We picked it through trial and error — a proper hyperparameter search would give a better threshold.
- One round of correction might not be enough for complex questions. We didn't implement multiple correction rounds because it would multiply the computation time.
- The token-overlap confidence score doesn't detect factual errors in numbers or names — the model could use the right words from the context but assemble them into a wrong claim, and the score wouldn't catch that.
- We also noticed that on very short answers (1–2 sentences), the coherence score drags down the overall confidence even when the answer is actually correct and well-grounded.

Possible Improvements

- Use a separate, stronger model (like Mistral-7B or a cross-encoder) for verification to reduce shared failure modes.
- Implement a multi-round correction loop with a maximum of 3 iterations and stop early if the score plateaus.

- Add NER-based checking to verify that specific values (drug names, numeric thresholds) in the answer match the source document exactly.
-

Task 5: System Evaluation, Limitations, and Encoder-Only Architecture Discussion

Marks: 2

Aim

The aim of this task is threefold: (i) to conduct a structured evaluation of the complete QA system across six qualitative dimensions; (ii) to identify and critically analyse the limitations of the system as observed during experimentation; and (iii) to provide a comprehensive architectural discussion of encoder-only, decoder-only, and encoder-decoder transformer models, demonstrating an understanding of their respective strengths, mechanisms, and applicability to NLP tasks.

```
# -----  
# TASK 5 – STEP 1: System Evaluation  
# Evaluate the pipeline across 6 quality dimensions using  
# scores derived from the Task 3 and Task 4 results.  
# -----  
  
# Compute per-question scores from verification_results  
eval_rows = []  
for i, (qr, vr) in enumerate(zip(qa_results, verification_results),  
1):  
    fc = vr["final_confidence"]  
    # Correctness: Combination of context overlap and whether  
    # correction was needed  
    correctness = round(fc["context_overlap"] * (1.0 if not  
vr["correction_triggered"] else 0.9), 2)  
    # Grounding: Context overlap score  
    grounding = fc["context_overlap"]  
    # Completeness: Length score  
    completeness = fc["length_score"]  
    # Clarity: Coherence score  
    clarity = fc["coherence_score"]  
    # Hallucination Reduction: improvement from correction (or  
    # baseline if no correction)  
    ic = vr["initial_confidence"]  
    fin = vr["final_confidence"]  
    hall_red = round(min(fin + 0.05, 1.0), 2) if
```



```

\ "What are the first-line medications for hyper... \ " \n      ], \n
\ "semantic_type \": \ " \", \n      \ "description \": \ " \ " \n      } \
n      }, \n      { \n      \ "column \": \ "Correctness \", \n
\ "properties \": { \n      \ "dtype \": \ "number \", \n      \ "std \":
0.0402492235949962, \n      \ "min \": 0.81, \n      \ "max \": 0.9, \n
\ "num_unique_values \": 2, \n      \ "samples \": [ \n      0.9, \n
0.81 \n      ], \n      \ "semantic_type \": \ " \", \n
\ "description \": \ " \ " \n      } \n      }, \n      { \n      \ "column \":
\ "Grounding \", \n      \ "properties \": { \n      \ "dtype \":
\ "number \", \n      \ "std \": 0.04472135954999578, \n      \ "min \":
0.9, \n      \ "max \": 1.0, \n      \ "num_unique_values \": 2, \n
\ "samples \": [ \n      1.0, \n      0.9 \n      ], \n
\ "semantic_type \": \ " \", \n      \ "description \": \ " \ " \n      } \
n      }, \n      { \n      \ "column \": \ "Completeness \", \n
\ "properties \": { \n      \ "dtype \": \ "number \", \n      \ "std \":
0.05406662556513029, \n      \ "min \": 0.075, \n      \ "max \": 0.2, \n
\ "num_unique_values \": 4, \n      \ "samples \": [ \n      0.2, \n
0.188 \n      ], \n      \ "semantic_type \": \ " \", \n
\ "description \": \ " \ " \n      } \n      }, \n      { \n      \ "column \":
\ "Clarity \", \n      \ "properties \": { \n      \ "dtype \": \ "number \", \n
\ "std \": 0.0, \n      \ "min \": 0.333, \n      \ "max \":
0.333, \n      \ "num_unique_values \": 1, \n      \ "samples \": [ \n
0.333 \n      ], \n      \ "semantic_type \": \ " \", \n
\ "description \": \ " \ " \n      } \n      }, \n      { \n      \ "column \":
\ "Hallucination Reduction \", \n      \ "properties \": { \n
\ "dtype \": \ "number \", \n      \ "std \": 0.026832815729997503, \n
\ "min \": 0.62, \n      \ "max \": 0.68, \n      \ "num_unique_values \":
3, \n      \ "samples \": [ \n      0.62 \n      ], \n
\ "semantic_type \": \ " \", \n      \ "description \": \ " \ " \n      } \
n      }, \n      { \n      \ "column \": \ "User Trust \", \n
\ "properties \": { \n      \ "dtype \": \ "number \", \n      \ "std \":
0.026832815729997458, \n      \ "min \": 0.56, \n      \ "max \": 0.62, \n
\ "num_unique_values \": 3, \n      \ "samples \": [ \n
0.56 \n      ], \n      \ "semantic_type \": \ " \", \n
\ "description \": \ " \ " \n      } \n      } \n      ] \
n } \, \ "type \": \ "dataframe \", \ "variable_name \": \ "eval_df" \}

```

Overall System Performance (Average Scores):

```

{ \ "summary \": { \n      \ "name \": \ "avg_df \", \n      \ "rows \": 6, \n      \ "fields \":
[ \n      { \n      \ "column \": \ "Parameter \", \n      \ "properties \": { \n
\ "dtype \": \ "string \", \n      \ "num_unique_values \": 6, \n
\ "samples \": [ \n      \ "Correctness \", \n      \ "Grounding \", \n
\ "User Trust \", \n      ], \n      \ "semantic_type \": \ " \", \n
\ "description \": \ " \ " \n      } \n      }, \n      { \n      \ "column \":
\ "Average Score (0 \u2013 1) \", \n      \ "properties \": { \n
\ "dtype \": \ "number \", \n      \ "std \": 0.31382275252122815, \n
\ "min \": 0.16019999999999998, \n      \ "max \": 0.9800000000000001, \n
\ "num_unique_values \": 6, \n      \ "samples \": [ \n      0.882, \n

```

```
0.9800000000000001,\n                0.6020000000000001\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n    },\n    {\n        \"column\": \"Observation\",\n        \"properties\": {\n            \"dtype\": \"string\",\n            \"num_unique_values\": 6,\n            \"samples\": [\n                \"Answers are largely factually aligned with the retrieved context\",\n                \"Strong context grounding due to RAG architecture\",\n                \"Composite trust score reflects reliable and grounded output\"\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"avg_df\"}
```

System Limitations

A critical analysis of the system's performance reveals the following limitations, each of which has practical implications for real-world deployment:

Limitation	Observed Impact	Mitigation Path
Weak Retrieval	Cosine similarity over dense embeddings may return topically adjacent but insufficiently specific chunks, particularly for questions involving rare medical entities not well-represented in the KB	Implement hybrid BM25 + dense retrieval with a cross-encoder reranker
Poor Prompts	Flan-T5's instruction-following is sensitive to prompt wording; minor variations in the constraint clause may result in off-context outputs	Systematic prompt evaluation and adversarial prompt testing
Ambiguous Questions	Single-word or underspecified queries (e.g., "Asthma") produce generalised and less clinically useful answers	Deploy question clarification dialogue before generation
Incomplete Documents	The synthetic knowledge base does not cover all medical conditions; out-of-scope queries produce low-confidence or evasive answers	Expand the KB with comprehensive clinical guidelines from authoritative sources (WHO, NICE)
Model Hallucination	Flan-T5 occasionally generates medically plausible but contextually unsubstantiated statements, particularly for procedural	Upgrade to a larger model; increase verification stringency

Limitation	Observed Impact	Mitigation Path
High Computation Cost	questions requiring causal reasoning	
	Loading three models (MiniLM, BART-MNLI, Flan-T5) simultaneously requires approximately 5–7 GB of memory, which may be prohibitive in constrained environments	Use quantised models (INT8/INT4) to reduce memory footprint
API Dependency	In production environments, deploying three separate Hugging Face models creates dependency on model availability and network bandwidth for initial downloads	Cache model weights locally; use ONNX-exported models for faster inference

Encoder-Only Architecture Discussion

1. Why Encoder-Only Models are Good at Understanding Language

Encoder-only models like BERT read the whole input at once — every word attends to every other word in both directions. This is called bidirectional attention. It's very different from decoder-only models (like GPT) which can only look at words to the left of the current position.

Why does this matter? Consider the sentence: *"The patient was treated with insulin, which controls blood glucose."* To understand what "which" refers to, you need to look both backward (to "insulin") and forward (to "controls blood glucose"). Decoder-only models can't do this because they only look backwards. Encoder-only models handle this naturally.

This makes encoder-only models much better at tasks where you need to deeply understand the meaning of an input — like classifying a question, finding similar sentences, or labelling medical terms.

2. How Bidirectional Attention Works

In the transformer self-attention mechanism, each word computes a query and compares it against keys from every other word. The formula is:

$$\alpha_{ij} = \text{softmax} \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right)$$

In encoder-only models, this runs across the full sequence — every word can attend to every other word. In decoder-only models, the attention is masked so each word can only see the words before it.

The output for each word is a weighted combination of all other words' value vectors:

$$h_i = \sum_{j=1}^n \alpha_{ij} v_j$$

So the final representation of each word already has the full sentence context baked into it. This is what makes BERT-style models so effective for understanding tasks.

3. Where Encoder-Only Models Are Useful

NLP Task	Why Encoder Helps	Example Model
Question Classification	Reads the whole question at once to understand intent	BERT, RoBERTa
Semantic Retrieval	Encodes sentences as dense vectors for similarity search	all-MiniLM-L6-v2
Extractive QA	Finds the start and end positions of the answer in a passage	BERT-QA
Named Entity Recognition	Tags each medical term (drug, disease, dosage) in context	BioBERT
Reranking	Jointly encodes question + document to score relevance	cross-encoder/ms-marco-MiniLM

In this project, the sentence transformer we used for retrieval (`all-MiniLM-L6-v2`) is a BERT-based encoder. It encodes both the question and each chunk into vectors, and we use cosine similarity to find the closest match.

4. Comparing the Three Architecture Types

Architecture	How It Reads Input	Training Goal	Best For	Not Good For	Examples
Encoder-Only	Both directions (full attention)	Predict masked words (MLM)	Classification, retrieval, NER, extractive QA	Text generation	BERT, RoBERTa, DistilBERT
Decoder-Only	Left to right only (causal attention)	Predict next word (CLM)	Text generation, few-shot tasks	Classification tasks that need full-context understanding	GPT-2/3/4, LLaMA, Mistral
Encoder-Decoder	Encoder: both directions; Decoder: left to right	Span corruption or denoising	Translation, summarisation, generative QA	Simple classification tasks (overkill)	T5, BART, Flan-T5

Decoder-only models for generation: They're very good at generating long, fluent text. But because they can't look at the full input bidirectionally, they're weaker at tasks like sentence

classification or understanding complex questions — where you need to process the entire input before making a decision.

Encoder-decoder models for sequence-to-sequence tasks: These are a good middle ground. The encoder reads the input fully (bidirectionally) and produces a rich representation. The decoder then uses that representation to generate the output one token at a time. This is why Flan-T5 works well for QA — it understands the question and context through the encoder, and generates the answer through the decoder. We used this in Tasks 3 and 4.

Inference — How Agentic Verification and Encoder-Only Models Improve QA Reliability

The experimental results of this assignment demonstrate two complementary mechanisms by which the reliability of domain-specific QA systems is enhanced:

1. Agentic Verification Improves Reliability Through Deliberative Self-Correction: The agentic verification loop addresses the fundamental limitation of single-pass generative models — the absence of any mechanism to detect or correct their own errors. By introducing a structured verification phase, the system transitions from a passive generator to an active quality-assurance agent. The experimental results show that initial answers with confidence scores below 0.65 consistently benefit from self-correction, with post-correction scores improving by measurable margins. Critically, the verification system is not merely cosmetic: it directly reduces the propagation of contextually unsupported claims to the end user, which in the medical domain is the difference between a useful clinical support tool and a dangerous misinformation vector.

2. Encoder-Only Models Improve Reliability Through Precise Grounding: The use of a sentence transformer (encoder-only architecture) for retrieval ensures that the generative model is grounded in the most semantically relevant portions of the knowledge base — not just the most lexically similar. This distinction is important in medical text, where synonym variation is common (e.g., "myocardial infarction" vs. "heart attack", "hypertension" vs. "high blood pressure"). The bidirectional attention of the encoder-only model enables it to capture these semantic equivalences that purely keyword-based retrieval would miss.

Together, encoder-only retrieval and agentic verification form a complementary architecture: the former ensures that the generative model is given the right context, and the latter ensures that the model stays within that context in its output. The combination significantly narrows the two primary sources of QA failure in specialised domains — retrieval gaps and generation hallucinations — and establishes a foundation for trustworthy, explainable, and clinically safe question answering.

Final Conclusion

This assignment walked through building a complete medical QA system from scratch — starting with a knowledge base, adding question classification, then answer generation, then verification, and finally evaluation.

Task 1 was about building the knowledge base and understanding why this domain needs to be handled carefully. The paragraph-level chunking worked better than we expected — the retrieval results were much more precise compared to when we initially tested with full documents.

Task 2 showed that combining rule-based and zero-shot classification gives a more robust classifier than either approach alone. The rule-based part handles the easy cases instantly, and the zero-shot model picks up the edge cases. The main gap we found is with paraphrased questions that don't match the expected patterns.

Task 3 validated the RAG approach. Flan-T5 followed the context constraint well for focused questions. For broader questions, the answers were generally correct but less tightly grounded, which is why verification is important.

Task 4 was the most interesting part for us. The self-correction loop actually improved answer quality in measurable ways for the lower-confidence cases. One observation we had: the corrected answers were sometimes shorter and more precise, which suggests the model was removing filler content that wasn't truly supported by the context — that's the right behaviour.

Task 5 brought everything together with evaluation and the architectural comparison. The encoder-only vs decoder-only vs encoder-decoder distinction was something we understood better after implementing this system — it's much clearer why retrieval needs a bidirectional model (encoder) while generation needs a sequential model (decoder).

Overall limitations we noticed through building this:

- The system works well within the 5 documents but completely fails on any medical topic not covered — there's no graceful handling for knowledge gaps beyond the "insufficient context" fallback message
- Running three models (MiniLM, BART, Flan-T5) together is heavy on memory — on our machine it took a few minutes just for initial model loading
- The confidence scoring is heuristic-based and can be gamed — a short, wrong answer that happens to use the same vocabulary as the context would score reasonably well
- We only tested on 5 questions per task; in practice, a broader test set would reveal more failure cases

The system is a reasonable proof-of-concept. For anything close to production use in a clinical setting, it would need a much larger and regularly updated knowledge base, a proper evaluation against ground-truth answers, and likely a larger generation model with better instruction following.

References

1. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.

2. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS 2020)*, 33, 9459–9474.
3. Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., ... & Le, Q. V. (2022). Finetuned Language Models are Zero-Shot Learners. *International Conference on Learning Representations (ICLR 2022)*.
4. Shinn, N., Cassano, F., Labash, B., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (NeurIPS 2023)*.